Trees and Dictionaries

Start Assignment

- Due Monday by 11:59pm
- Points 20
- Submitting a file upload

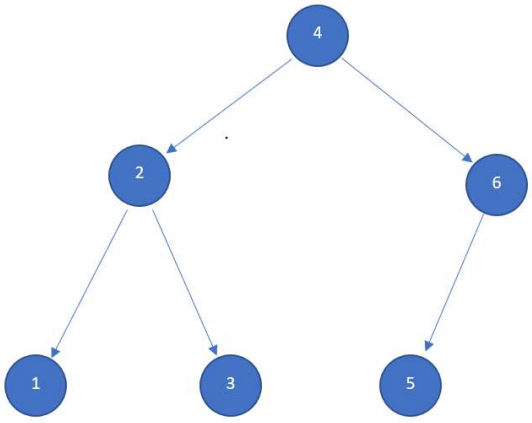
Goal

This assignment will have you complete two separate programs. The first will have you construct and perfom operations on a tree. The second will have you make use of a dictionary to look-up information.

Trees

Recall that a tree has a root label and a sequence of branches. Each branch of a tree is a tree. A tree with no branches is called a leaf. Any tree contained within a tree is called a sub-tree of that tree (such as a branch of a branch). The root of each sub-tree of a tree is called a node in that tree. The data abstraction for a tree includes a constructor for the tree and the selectors label and branches, as well as a function to count the number of leaves in the tree.

Write a driver program that makes use of the appropriate functions to construct the following tree with nested expressions:



Then have the program print out the nested tree, its label and branches, and the number of leaves it has.

Name the file `lastname_firstname_bTree.py`

Dictionaries

DNA is made up of sequences of nucleotides. There are four nucleotides: Adenine, Cytosine, Guanine, and Thymine, represented by the letters A, C, G, and T.

Every set of three nucleotides produces one of twenty amino acids. For example, Cytosine, Thymine, and Guanine (CTG) produce Leucine. Note that there are 64 combinations of three nucleotides, and only twenty amino acids, so that means that a single amino acid can be produced in many different ways.

The following tables, taken from the **BioInformatics tutorial** (https://www.genzentrum.uni-muenchen.de/funktionen/fehler/404.html) are Copyright 1999 by Boris Steipe, University of Munich. The table at the left shows the names of the amino acids and their abbreviations. The table at the right shows the abbreviations of the amino acids and the nucleotide sequences which produce them.

"T"	"C"	"G"	"A"
TTT:Phe	TCT:Ser	TGT:Cys	TAT:Tyr
TTC:Phe	TCC:Ser	TGC:Cys	TAC:Tyr
TTG:Leu	TCG:Ser	TGG:Trp	TAG:***
TTA:Leu	TCA:Ser	TGA:***	TAA:***
-	-	-	-
CTT:Leu	CCT:Pro	CGT:Arg	CAT:His
CTC:Leu	CCC:Pro	CGC:Arg	CAC:His
CTG:Leu	CCG:Pro	CGG:Arg	CAG:Gln
CTA:Leu	CCA:Pro	CGA:Arg	CAA:Gln
-	-	-	-
GTT:Val	GCT:Ala	GGT:Gly	GAT:Asp
GTC:Val	GCC:Ala	GGC:Gly	GAC:Asp
GTG:Val	GCG:Ala	GGG:Gly	GAG:Glu
GTA:Val	GCA:Ala	GGA:Gly	GAA:Glu
-	-	-	-
ATT:Ile	ACT:Thr	AGT:Ser	AAT:Asn
ATC:Ile	ACC:Thr	AGC:Ser	AAC:Asn
ATG:Met	ACG:Thr	AGG:Arg	AAG:Lys
ATA:Ile	ACA:Thr	AGA:Arg	AAA:Lys

You will write a program that repeatedly accepts as input a string of ACGT triples and produces a list of the triples and the corresponding amino acids, one set per line.

The program will continue to accept input until the user just presses ENTER without entering any DNA codes.

Here is a sample run of the program. Note that some sequences, such as TAG, do not produce an amino acid. They are listed as `***` for their three-letter abbreviation. Do not presume that there will never be more than 4 sequences. Somebody could enter a string with 40 three-letter sequences, or 50, or 80, and your program would have to process that string correctly.

```
Enter a nucleotide sequence, or just press ENTER to quit: aaacgatcaccc
AAA Lys
CGA Arg
TCA Ser
CCC Pro

Enter a nucleotide sequence, or just press ENTER to quit: cga tgc
CGA Arg
TGC Cys

Enter a nucleotide sequence, or just press ENTER to quit: cgi tat tca
CGI invalid sequence
TAT Tyr
TCA Ser

Enter a nucleotide sequence, or just press ENTER to quit: tag cat agg ctt
TAG ***
CAT His
AGG Arg
CTT Leu

Enter a nucleotide sequence, or just press ENTER to quit: tca cag gtt cg
Error: You must give complete triples.

Enter a nucleotide sequence, or just press ENTER to quit:
```

Breaking a String into Triples

This takes a bit of doing. I suggest you do this:

- Write a function named `clean_sequence()` takes a string as its single parameter. It will use a `for` loop to go through the string one character at a time; if the letter is alphabetic (hint: `isalpha()`) , convert it to upper case and add it on to the end of a result string. The function returns the result string. For example, `clean_sequence('aaa, TCG, gaa')` will return `'AAATCGGAA'`
- Use a `while` loop that goes through the cleaned sequence one triple at a time (hint: use slices) and processes it.

Program Requirements

- You *must* use at least one dictionary to implement this program.

- If you get an invalid sequence (one with a letter other than A,C,G, or T), print “invalid sequence” or some other meaningful message in your list, as shown in the sample run.
- You must provide an error message if the string has extra or missing letters, such as `ACTGA` or `CA`. (Hint: use the length of `sequence` after you have gotten rid of non-letters with the code above.)

When You Finish

Name the file `lastname_firstname_dna.py` . **Save the source code** `lastname_firstname_dna.py` **Take the all screenshot and save in to Microsoft word file name Wk7LabScreenshot.doc**

Upload the above source code and test program source files and Wk7LabScreenshot.doc)

Trees and Dictionaries

Criteria	Ratings			Pts
Construction of Tree	**3 pts** **Full Marks - Is Correct**	**2 pts** **Partial credit**	**0 pts** **No Marks**	3 pts
Printed label	**1 pts** **Full Marks - Is correct**		**0 pts** **No Marks**	1 pts
Branches	**1 pts** **Full Marks**		**0 pts** **No Marks**	1 pts
Number of leaves	**2 pts** **Full Marks - Is correct**		**0 pts** **No Marks**	2 pts
Uses dictionary	**2 pts** **Full Marks - Has dictionary**		**0 pts** **No Marks**	2 pts
Non-letters in input is ignored	**2 pts** **Full Marks - Yes**		**0 pts** **No Marks**	2 pts
Requires exact triples	**2 pts** **Prints error if input is not a multiple of three**	**1 pts** **Doesn't process string, but fails to print error message**	**0 pts** **No Marks**	2 pts
Finds correct amino acids	**2 pts** **Does dictionary lookup correctly**		**0 pts** **No Marks**	2 pts
Error message for non-ACGT	**2 pts** **Full Marks**		**0 pts** **No Marks**	2 pts
Repeated input	**2 pts** **Full Marks**		**0 pts** **No Marks**	2 pts
Exit on empty input	**1 pts** **Full Marks**		**0 pts** **No Marks**	1 pts
			Total Points: 20	